CSE 2246
PS#2

2.3

2. Find the order of growth of the following sums. Use the $\Theta(g(n))$ notation with the simplest function $g(n)$ possible.

a. $\sum_{i=0}^{n-1}(i^2+1)^2$ **b.** $\sum_{i=2}^{n-1} \lg i^2$

c. $\sum_{i=1}^{n}(i+1)2^{i-1}$ **d.** $\sum_{i=0}^{n-1}\sum_{j=0}^{i-1}(i+j)$

d) $\sum_{i=0}^{n-1}\sum_{j=0}^{i-1}(i+j) = \sum_{i=0}^{n-1}\left(\sum_{j=0}^{i-1}(i+j)\right)$

$= \sum_{i=0}^{n-1}\left[i\cdot(i-1-0+1) + [0+1+2+\cdots+i-1]\right]$

$[0+1+2+\cdots+i-1] = \frac{(i-1)\cdot(i-1+1)}{2}$

$= \sum_{i=0}^{n-1}\left[i^2 + \frac{i^2-i}{2}\right] = \sum_{i=0}^{n-1}\frac{2i^2+i^2-i}{2}$

$= \sum_{i=0}^{n-1}\frac{3i^2-i}{2} = \sum_{i=0}^{n-1}\frac{3}{2}i^2 - \sum_{i=0}^{n-1}\frac{i}{2}$

$= \frac{3}{2}\sum_{i=0}^{n-1}i^2 - \frac{1}{2}\sum_{i=0}^{n-1}i$

$\sum_{i=0}^{n-1}i = \frac{(n-1)\cdot(n-1+1)}{2}$

$\sum_{i=0}^{n-1}i^2 = \frac{(n-1)\cdot(n-1+1)\cdot(2n-2+1)}{6}$

$= \frac{3}{2}\cdot\frac{(n-1)\cdot(n-1+1)\cdot(2n-2+1)}{6} - \frac{1}{2}\cdot\frac{(n-1)\cdot(n-1+1)}{2}$

$$= \frac{\overbrace{(n-1)(n)(2n-1)}^{n^3}}{4} - \frac{1}{2} \cdot \frac{\overbrace{(n-1).n}^{n^2}}{2}$$

$$= \Theta(n^3) + \Theta(n^2) = \Theta(n^3)$$

2.3

4. Consider the following algorithm.

ALGORITHM *Mystery*(n)

//Input: A nonnegative integer n

$S \leftarrow 0$

for $i \leftarrow 1$ **to** n **do**

  $S \leftarrow S + i * i$

return S

a. What does this algorithm compute?

b. What is its basic operation?

c. How many times is the basic operation executed?

d. What is the efficiency class of this algorithm?

e. Suggest an improvement, or a better algorithm altogether, and indicate its efficiency class. If you cannot do it, try to prove that, in fact, it cannot be done.

$n = 5$

a) $S = 0$

$i = 1$

$S = 0 + 1 \times 1 = 1$

$i = 2$

$S = 1 + 2 \times 2 = 1 + 2^2$

$i = 3$

$S = \underbrace{1 + 2^2}_{S_{prev}} + 3 \cdot 3 = 1 + 2^2 + 3^2$

$i = 4$

$S = 1 + 2^2 + 3^2 + 4 \cdot 4$

$\vdots$

$S = 1 + 2^2 + 3^2 + 5^2 + \dots + n^2$

$$S(n) = \sum_{i=0}^{n} i^2$$

b) Basic Op: Multiplication

c) $\sum_{i=1}^{n} 1 = n$

d) $\theta(n)$

e) $\theta(n) \longrightarrow \theta(1)$

$\sum_{i=1}^{n} i^2 \rightarrow \frac{n(n+1)(2n+1)}{6} \rightarrow \theta(1)$

2.3

6. Consider the following algorithm.

ALGORITHM *Enigma*$(A[0..n-1, 0..n-1])$

```
//Input: A matrix A[0..n − 1, 0..n − 1] of real numbers
for i ← 0 to n − 2 do
    for j ← i + 1 to n − 1 do
        if A[i, j] ≠ A[j, i]
            return false
return true
```

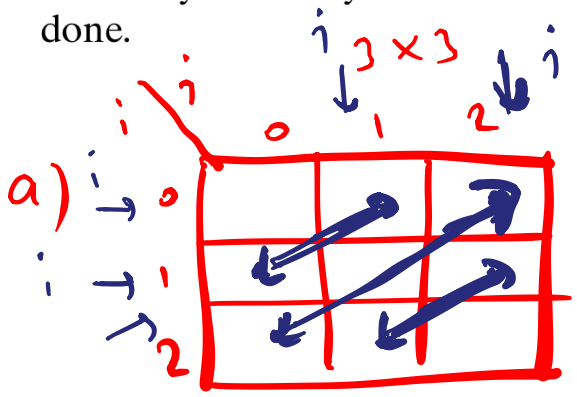
a. What does this algorithm compute?

b. What is its basic operation?

c. How many times is the basic operation executed?

d. What is the efficiency class of this algorithm?

e. Suggest an improvement, or a better algorithm altogether, and indicate its efficiency class. If you cannot do it, try to prove that, in fact, it cannot be done.



$A[0,1] \overset{?}{=} A[1,0]$

$A[0,2] \overset{?}{=} A[2,0]$

$A[1,2] = A[1,1]$

check whether a given array is Symmetric or not

b) Basic op: Comparison

c) $\sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [n-1-i-1+1]$

$= \sum_{i=0}^{n-1} (n-1-i)$

$$= \sum_{i=0}^{n-1} n-1 - \sum_{i=0}^{n-1} i$$

$$= (n-1)[n-1-0+1] - \frac{(n-1).(n-1+1)}{2}$$

$$= n^2 - n - \frac{n^2-n}{2}$$

$$= \frac{2n^2 - 2n - n^2 + n}{2} = \frac{n^2 - n}{2}$$

$C_b(n) = \Theta(1)$ → best case

$C_w(n) = \Theta(n^2)$ → worst case

$C(n) = O(n^2)$ → general

e) Optimal, Do not reduce time complexity

2.4

1. Solve the following recurrence relations.

a. $x(n) = x(n-1) + 5$ for $n > 1$, $x(1) = 0$

b. $x(n) = 3x(n-1)$ for $n > 1$, $x(1) = 4$

c. $x(n) = x(n-1) + n$ for $n > 0$, $x(0) = 0$

d. $x(n) = x(n/2) + n$ for $n > 1$, $x(1) = 1$ (solve for $n = 2^k$)

e. $x(n) = x(n/3) + 1$ for $n > 1$, $x(1) = 1$ (solve for $n = 3^k$)

a) $x(n) = x(n-1) + 5$, $x(1) = 0$

$x(n-1) = x(n-2) + 5$

$= x(n-2) + 5 + 5$

$x(n-2) = x(n-3) + 5$

$= x(n-3) + 5 + 5 + 5$ (3. 5)

$x(n-3) = x(n-4) + 5$

$= x(n-4) + 5 + 5 + 5 + 5$

$4 \cdot 5$

$\vdots$

$= x(n-i) + i \cdot 5$ (i up to $n-1$)

$= x(n-(n-1)) + (n-1) \cdot 5$

$= x(n-n+1) + (n-1) \cdot 5$

$= x(1) + (n-1) \cdot 5$ (since $x(1) = 0$)

$x(n) = 5(n-1)$

b) $x(n) = 3x(n-1)$, $x(1) = 4$

$x(n-1) = 3x(n-2)$

$= 3[3x(n-2)] = 3^2 \cdot x(n-2)$

$x(n-2) = 3 \cdot x(n-3)$

$= 3^2[3 \cdot x(n-3)]$

$= 3^3 x(n-3)$

$\vdots$

$= 3^i x(n-i)$

$= 3^{n-1} x(n-n+1)$

$= 3^{n-1} x(1)$

$= 4$

$x(n) = 4 \cdot 3^{n-1}$

c) $x(n) = x(n-1) + n$, $x(0) = 0$

$x(n-1) = x(n-2) + n-1$

$= x(n-2) + n-1 + n$

$x(n-2) = x(n-3) + n-2$

$= x(n-3) + (n-2) + (n-1) + n$

$\vdots$

$= x(n-i) + (n-(i-1)) + (n-(i-2)) + \cdots + n$

$= x(n-n) + (n-n+1) + (n-n+2) + \cdots + n$

$= \underbrace{x(0)}_{0} + \underbrace{1 + 2 + \cdots + n}$

$$x(n) = \frac{n(n+1)}{2}$$

d) $x(n) = x(n/2) + n$, $x(1) = 1$

$n = 2^k$

$$x(2^k) = x(2^{k-1}) + 2^k$$

$$x(2^{k-1}) = x(2^{k-2}) + 2^{k-1}$$

$$= x(2^{k-2}) + 2^{k-1} + 2^k$$

$$x(2^{k-2}) = x(2^{k-3}) + 2^{k-2}$$

$$= x(2^{k-3}) + 2^{k-2} + 2^{k-1} + 2^k$$

$$= x(2^{k-i}) + 2^{k-(i-1)} + 2^{k-(i-2)} + \dots + 2^k$$

$$= x(2^{k-k}) + 2^{k-(k-1)} + 2^{k-(k-2)} + \dots + 2^k$$

$$= x(2^0) + 2^1 + 2^2 + \dots + 2^k$$

$$= \underset{x(1)}{1} + 2^1 + 2^2 + \dots + 2^k$$

$$= 2^{k+1} - 1/2 - 1$$

$x(2^k) = 2^{k+1} - 1 = 2 \cdot 2^k - 1$

n

$$x(n) = 2n - 1$$

$n = 3^k$

e) $x(n) = x(n/3) + 1$, $x(1) = 1$

$x(3^k) = x(3^{k-1}) + 1$

$x(3^{k-1}) = x(3^{k-1-1}) + 1$

$= x(3^{k-2}) + 1 + 1$

$x(3^{k-2}) = x(3^{k-2-1}) + 1$

$= x(3^{k-3}) + 1 + 1 + 1$

$\vdots$

k

$= x(3^{k-i}) + i$

$= x(3^{k-k}) + k$

$= x(1) + k = 1 + k$

$\log_3 n$

$x(n) = 1 + \log_3 n$

2.4

3. Consider the following recursive algorithm for computing the sum of the first n cubes: $S(n) = 1^3 + 2^3 + \cdots + n^3$.

ALGORITHM $S(n)$

//Input: A positive integer n
//Output: The sum of the first n cubes
if $n = 1$ **return** 1
else return $S(n-1) + n * n * n$

a. Set up and solve a recurrence relation for the number of times the algorithm's basic operation is executed.

b. How does this algorithm compare with the straightforward nonrecursive algorithm for computing this sum?

a) Basic op. : multiplication

$M(n) = M(n-1) + 2$, $M(1) = 0$

$M(n-1) = M(n-2) + 2$

$= M(n-2) + 2 + 2$

$M(n-2) = M(n-3) + 2$

$= M(n-3) + 2 + 2 + 2$

$\vdots$

$= M(n-i) + 2i$, $i = n-1$

$= M(1) + 2(n-1) = 2n-2$

$M(1) = 0$

b) Algorithm MonRec(n)

$S \leftarrow 1$

else
for $i = 2$ to n do
$S \leftarrow S + i \times i \times i$

return S

$n = 3 \rightarrow i = 2$

$S = 1 + 2.2.2 = 1 + 2^3$

$i = 3 \quad S = 1 + 2^3 + 3 \times 3 \times 3$

$S = 1 + 2^3 + 3^3$

return S

Basic Op: multiplication

$\#$ multiplications in each round

$$\sum_{i=2}^{n} 2 = 2.[n-2+1] = 2(n-1)$$

$$= 2n - 2$$

2.4

4. Consider the following recursive algorithm.

ALGORITHM $Q(n)$

//Input: A positive integer n
if $n = 1$ **return** 1
else return $Q(n-1) + 2 * n - 1$

a. Set up a recurrence relation for this function's values and solve it to determine what this algorithm computes.

b. Set up a recurrence relation for the number of multiplications made by this algorithm and solve it.

c. Set up a recurrence relation for the number of additions/subtractions made by this algorithm and solve it.

a) $\Theta(n)$: function's values

$\Theta(n) = \Theta(n-1) + 2n - 1$, $\Theta(1) = 1$

$\Theta(2) = \underbrace{\Theta(1)}_{1} + 2 \cdot 2 - 1 = 4 = 2^2$

$\Theta(3) = \underbrace{\Theta(2)}_{4} + 2 \cdot 3 - 1 = 9 = 3^2$

$\Theta(4) = \underbrace{\Theta(3)}_{9} + 2 \cdot 4 - 1 = 16 = 4^2$

$$\boxed{\Theta(n) = n^2}$$

b) $M(n) = M(n-1) + 1$, $M(1) = 0$

$M(n-1) = M(n-2) + 1$

$= M(n-2) + 1 + 1$

$M(n-2) = M(n-3) + 1$

$= M(n-3) + 1 + 1 + 1$

$\vdots$

$= M(n-i) + i$

$= M(n-(n-1)) + n - 1$

$= M(1) + n - 1$

$= 0$

$M(n) = n - 1$

c) $A(n) = A(n-1) + 3$, $A(1) = 0$

$A[n-1] = A(n-2) + 3$

$= A(n-2) + 3 + 3$

$A(n-2) = A(n-3) + 3$

$= A(n-3) + 3 + 3 + 3$

$\vdots$

$= A(n-i) + i \cdot 3$

$= A(n-(n-1)) + (n-1) \cdot 3$

$= A(n-n+1) + (n-1) \cdot 3$

$= A(1) + (n-1) \cdot 3$

$A(1) = (n-1) \cdot 3$

2.4

8. a. Design a recursive algorithm for computing 2^n for any nonnegative integer n that is based on the formula $2^n = 2^{n-1} + 2^{n-1}$.

b. Set up a recurrence relation for the number of additions made by the algorithm and solve it.

c. Draw a tree of recursive calls for this algorithm and count the number of calls made by the algorithm.

d. Is it a good algorithm for solving this problem?

a) Algorithm RecPow(n)

IF n = 0
return 1
else
return RecPow(n-1) + RecPow(n-1)

b) $A(n) = 2 \cdot A(n-1) + 1$, $A(0) = 0$

$A(n-1) = 2 \cdot A(n-2) + 1$

$= 2[2 \cdot A(n-2) + 1] + 1$

$= 2^2 \cdot A(n-2) + 2 + 1$

$A(n-2) = 2A(n-3) + 1$

$= 2^2[2 \cdot A(n-3) + 1] + 2 + 1$

$= 2^3 \cdot A(n-3) + 2^2 + 2 + 1$

$$= 2^i A(n-i) + 2^{i-1} + 2^{i-2} + \cdots + 2 + 1$$

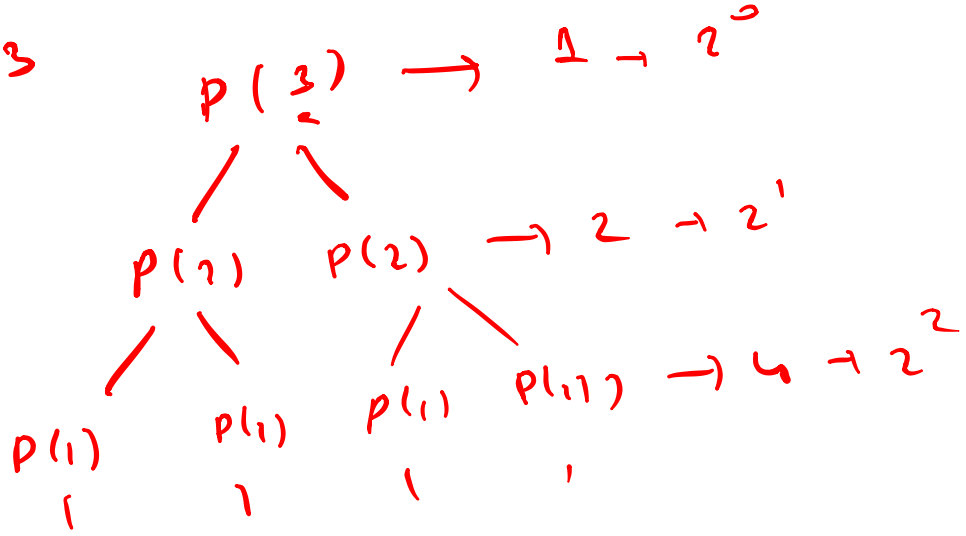
$$= 2^n \cdot A(n-n) + 2^{n-1} + 2^{n-2} + \cdots + 2 + 1$$

$$= 2^{n-1} + 2^{n-2} + \cdots + 2 + 1$$

$$A(n) = \frac{2^{n-1+1} - 1}{2-1} = 2^n - 1$$

---

c) $n = 3$



$$1 + 2 + 2^2 + \cdots + 2^n = \frac{2^{n+1} - 1}{2-1} = 2^{n+1} - 1$$

$$\Theta(2^n)$$

d) Can we improve it ?

Algorithm NonRec(n)
// n is a nonnegative integer

res ← 1

for i = 1 to n do

res ← res * 2

return res

For n = 3

i	res
1	1 * 2 = 2
2	2 * 2 = 4
3	4 * 2 = 8 = 2^3